ACIT 4610

Final Group Project -2026

Purpose: Implement and study the application of algorithms/techniques discussed in this course.

Submission Deadline: November 30 (Monday) at 12:00 (mid-day).

Submission Channel: Inspera.

- Each group must submit **ONLY ONE final version**, submitted by any member of the group.
- The Inspera submission will be available approximately one week prior to the deadline.
- We will download the code immediately after the submission deadline to prevent any subsequent updates.

Group Policy: Pre-approved groups only. Submissions outside the group list will NOT be accepted.

Use of Artificial Intelligence (AI):

- You may use AI tools for coding as permitted by university guidelines, but NO AI tools are permitted to write the report.
- **In the report, state whether AI tools were used, what they were used for, and approximately what percentage of the total code was generated by AI.**
- **You must clearly mark the AI-produced code (if used).**
- You can find the OsloMet policy at: https://student.oslomet.no/en/veiledning-kunstig-intelligens-studentoppgaver

Cheating / Plagiarism: Any cheating/plagiarism will be handled by OsloMet's policy. You can find more about it at https://student.oslomet.no/en/cheating

Prerequisite for the Submission:

The final submission is not something you can complete in a single day. As part of the grading process, your group activities will be monitored. To meet the requirements for the final submission, you must address the following criteria:

- Provide one shareable GitHub repository that will contain the final submissions of this Portfolio. Create and share this GitHub repository with me during week 44 (26 October-1 November 2026), and grant me access so I can follow up on gradual improvements and facilitate group collaboration.
- Send a weekly group report including:
  - Percentages and activity lists for each member.
  - Planned activities for the upcoming week.
- Demonstrating actual improvement in implementation in the weekly group report is not mandatory.

Assignments

There are **Two (02)** problems in this Portfolio. You need to implement **all** these problems as per the requirements and instructions mentioned in each problem.

Deliverables

a. **Code Implementation:**
   i. Provide one shareable GitHub repository containing clean Python code, executable scripts, step-by-step installation instructions in `README.md`, and inline documentation

explaining your algorithmic logic. The `README` must include instructions for installing any dependencies and running the algorithms on the provided example data.

ii. The code must be executable and generate the outputs.

iii. The code must load the required data files.

iv. The repository must contain executable scripts/notebooks that reproduce the reported experiments and figures.

v. Include the GitHub link in the submitted report and ensure the repository is accessible and the code can be run.

b. **Report**

A combined report covering all the requirements outlined in the two problems. The report must meet the following criteria:

i. The combined report should be between 3,000 and 7,500 words in total.

ii. The report must be submitted in PDF format.

iii. The report must mention the Group No as published in Canvas.

iv. The report should contain the names of all group members.

v. The report should include the link to the group repository, with a separate folder for each problem.

vi. Do not include the code within the report.

Problem-1

Overview

Use the Bees Algorithm (BA) to solve a discrete route-planning problem (**Part-A**) and continuous optimization problems (**Part-B**). Complete both parts and evaluate solution quality, convergence, and consistency across repeated runs.

Bees Algorithm (BA)

Implement the BA introduced by Pham and colleagues [1] and discussed in lectures. Use a permutation of cities in Part A and a vector of real numbers in Part B. Numerical, data-loading, and plotting libraries may be used. Choose and record the BA settings for each part.

Use 30,000 objective evaluations per run. Count the initial population, recruited neighbors, and fresh scouts, including candidates that are rejected. Stop at the budget, allowing an incomplete final iteration. Record the initial best value, the best-so-far history, and the final best solution.

Part A: Discrete Route Planning

The Travelling Salesman Problem

Imagine a vehicle leaving a depot, making a delivery at every location, and returning to the depot. The locations remain the same; the decision is the order in which to visit them. A different order changes the total distance travelled. Find the shortest complete round trip.

This is the symmetric Traveling Salesman Problem (TSP): visit every city exactly once and return to the starting city. The distance between any two cities is the same in either direction. Here, distances are derived from benchmark coordinates rather than from a road network.

Represent a solution as a permutation $\pi = (\pi_1, \ldots, \pi_n)$ of the city identifiers. Minimize

$$L(\pi) = \sum_{i=1}^{n-1} d_{\pi_i,\pi_{i+1}} + d_{\pi_n,\pi_1}.$$

The permutation lists each city once. The objective sums all consecutive legs and the return leg. Fix city 1 as the first city; append it only when displaying the closed route, not as a second entry in the permutation.

Data and Distance Calculation

Use all six TSPLIB instances below from Heidelberg University [2]. Each filename links to the official compressed file; decompress it before loading. TSPLIB [3] publishes the optimal lengths and provides numerical accuracy references.

Instance and download	Cities	Distance type	Optimal tour length
eil51.tsp.gz	51	EUC_2D	426
berlin52.tsp.gz	52	EUC_2D	7,542
st70.tsp.gz	70	EUC_2D	675
eil76.tsp.gz	76	EUC_2D	538
kroA100.tsp.gz	100	EUC_2D	21,282
ch150.tsp.gz	150	EUC_2D	6,528

Validate NAME, TYPE, DIMENSION, EDGE_WEIGHT_TYPE, and NODE_COORD_SECTION. Require TYPE = TSP, EDGE_WEIGHT_TYPE = EUC_2D, and the stated number of rows with unique city identifiers.

- Use TSPLIB's rounded Euclidean distances [4]. For city coordinates (a_i, b_i) and (a_j, b_j), calculate

$$d_{ij} = \text{floor}\left(\sqrt{(a_i - a_j)^2 + (b_i - b_j)^2} + 0.5\right).$$

Use the TSPLIB rule above [4]: floor means round downward. Round each edge before adding the tour length, including the return edge. Validate that every returned tour contains every city once. Use published optimal tours only to check distances, never to initialize or guide BA.

BA Representation and Neighborhoods

Initialize scouts with uniformly shuffled permutations. Implement two BA versions: one swaps two city positions, and the other reverses a segment between two positions. Use a single randomly selected move to create each recruited neighbor, leaving city 1 fixed. Both moves preserve a valid permutation.

Use the same BA settings and seed list for the two neighborhood versions so that their behavior can be interpreted under the same evaluation budget.

Experiments and Required Results

- **Runs:** Use 10 independent seeds for each of the six instances and each neighborhood version, giving 120 TSP runs.
- **Metrics:** Record the final tour length, percentage gap, runtime, and evaluation count.

$$\text{Gap}(\%) = 100\,\frac{L_{BA} - L^*}{L^*}.$$

Here, L^* is the optimal length in the dataset table and L_{BA} is the final best valid tour length.

- **Summary table:** For each instance and neighborhood version, report the instance name, cities, optimum, best, mean, and standard deviation of the final tour length, mean percentage gap, and mean runtime. Split tables if needed for readability.
- **Convergence plots:** For `berlin52` and `ch150`, plot evaluations against the mean best-so-far tour length across the 10 runs. Show both neighborhood versions on each plot, with the optimal length marked.

- **Route plots:** Show the shortest valid `berlin52` tour found across both neighborhood versions and all runs. Label the neighbourhood, seed, tour length, percentage gap, and starting city, and draw the return leg.

Part B: Continuous Function Optimization

Problem Description

Use BA to find a vector of real numbers that minimizes a benchmark function. Each coordinate is a decision variable constrained to a specified interval. Unlike a TSP move, a continuous move changes numerical values rather than the order of cities. Run each function at dimensions 10 and 30.

Functions and Bounds

Use the following four unshifted, noiseless functions from Surjanovic and Bingham's Simon Fraser University collection [5–8]. Evaluate the formulas directly; no external dataset is required. D denotes the number of coordinates. The symbols 0 and 1 below denote vectors with all coordinates zero or all coordinates one.

Sphere [5]:

$$f(\mathbf{x}) = \sum_{i=1}^{D} x_i^2.$$

Bounds: $x_i \in [-5.12, 5.12]$. Global minimum: $f(\mathbf{0}) = 0$. This is a convex, unimodal function.

Rosenbrock [6]:

$$f(\mathbf{x}) = \sum_{i=1}^{D-1} [100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2].$$

Bounds: $x_i \in [-5,10]$. Global minimum: $f(\mathbf{1}) = 0$. Its narrow-curved valley makes progress toward the optimum difficult.

Rastrigin [7]:

$$f(\mathbf{x}) = 10D + \sum_{i=1}^{D} [x_i^2 - 10\cos(2\pi x_i)].$$

Bounds: $x_i \in [-5.12, 5.12]$. Global minimum: $f(\mathbf{0}) = 0$. It has many local minima.

Ackley [8]:

$$f(\boldsymbol{x}) = -20 \cdot \exp\left(-0.2\sqrt{\frac{1}{D}\sum_{i=1}^{D} x_i^2}\right) - \exp\left(\frac{1}{D}\sum_{i=1}^{D} \cos(2\pi x_i)\right) + 20 + \exp(1)$$

Bounds: $x_i \in [-32.768, 32.768]$. Global minimum: $f(\mathbf{0}) = 0$. It has many local minima and a relatively flat outer region.

Check each Python function at its known minimizer and at one other point against the Simon Fraser University (SFU) definitions or reference code. Allow a small floating-point tolerance. Running MATLAB or R is not required. Do not insert known minimizing vectors into the search.

BA Representation and Neighborhood

Initialize scouts uniformly within the bounds. For each coordinate, independently draw a random change uniformly between −10% and +10% of its allowed range (upper bound minus lower bound). Add the change and clip the result to the bounds. Record the settings used for site selection, recruitment, and scouting.

Experiments and Required Results

- **Runs:** Use 10 independent seeds for each function at D = 10 and 30: 8 cases and 80 runs. Complete the evaluation budget in every run.
- **Metrics:** Record the final best objective value, the corresponding vector, the runtime, and the number of evaluations used. Since the global minimum is zero, the final objective value is the objective error, except for numerical rounding.
- **Summary table:** For all 8 cases, report the function, dimension, best, mean, and standard deviation of the final objective values, and mean runtime.
- **Convergence plots:** For **Rastrigin** at D = 10 only, plot evaluations against the mean best-so-far objective across 10 runs. Label any axis transformation.
- **Final-result plots:** Show boxplots of final objectives for the four functions at D = 30 as four panels in one figure, with labeled axes and captions.

Deliverables

- **Code**
  - **Implementations:** BA for TSP using swap and segment reversal, BA for continuous optimization, TSPLIB loading and tour validation, correct distance and function calculations, and bounds handling.
  - **Reproduction scripts:** Commands or scripts for all required runs, summary tables, convergence plots, route plots, and boxplots. Include raw per-run results and best-so-far logs, with seeds, objective values, evaluation counts, runtime, and final tours or vectors.
  - **Data and `README`:** Include the six original TSPLIB instances and their source URLs. The `README` must give dependency versions, all BA settings, seed lists, data locations, and exact commands for one run and the full experiment set.

- **Report**
  - **Problem and data:** Explain both optimization problems. State the exact TSPLIB filenames and optima, distance rule, function definitions, bounds, dimensions, and validation checks.
  - **Implementation:**
    - Describe site selection, elite recruitment, local search, retained solutions, scouting, and stopping.
    - Explain the representations, neighborhood moves and bounds handling. Give implementation settings in the `README`.
  - **Results:**
    - Include every table and plot required in Parts A and B, with repeated-run statistics, units, legends, and captions.
    - Identify the seeds used for illustrated routes and explain how gaps were calculated.

- **Discussion:**
  - Explain improvement from initial solutions, convergence, run-to-run variability, and unsuccessful cases.
  - Discuss the two TSP neighborhoods, larger instances, and higher dimensions. Explain what changes between discrete and continuous BA and what remains common.

References and Data Sources

The six TSPLIB files, optimal lengths, and SFU function definitions and implementations were checked on 20 October 2026.

1. Pham, D. T., and colleagues. **The Bees Algorithm — A Novel Tool for Complex Optimisation Problems**. Intelligent Production Machines and Systems, 2006, pp. 454–459. Publisher record.
2. Reinelt, G. **TSPLIB**. Heidelberg University. Library home page.
3. Reinelt, G. **Optimal solutions for symmetric TSPs**. Reference tour lengths.
4. Reinelt, G. **TSPLIB 95 documentation**, especially Sections 1 and 2.1 on file format and Euclidean distances. Documentation PDF.
5. Surjanovic, S., and Bingham, D. **Sphere Function**. Simon Fraser University. Function and implementation.
6. Surjanovic, S., and Bingham, D. **Rosenbrock Function**. Simon Fraser University. Function and implementation.
7. Surjanovic, S., and Bingham, D. **Rastrigin Function**. Simon Fraser University. Function and implementation.
8. Surjanovic, S., and Bingham, D. **Ackley Function**. Simon Fraser University. Function and implementation.

Problem-2

Portfolio Optimization via Evolution Strategies (ES)

In this assignment, you will use ES to allocate an investment budget across several assets, such as shares in different companies. The goal is to find an allocation that minimises portfolio risk while meeting a minimum expected return. You will implement three ES variants ((1+1)ES, (μ+λ)-ES, and (μ,λ)-ES) to minimize portfolio variance while satisfying the budget, non-negativity, and minimum-return constraints. You will also evaluate and compare their solution quality, convergence, consistency, and runtime across the 15 test cases.

You must complete all required ES implementations, benchmark experiments, and empirical analyses outlined in this specification.

Problem Formulation

An investment manager must divide a fixed budget across ***n* assets**. Each asset has an estimated return and a distinct level of risk. The decision is how much of the budget to invest in each asset so that the combined portfolio has the lowest possible risk while meeting the required expected return.

A simple example: Suppose the budget is NOK 100,000 and there are three assets. Allocating 50%, 30%, and 20% means investing NOK 50,000, NOK 30,000, and NOK 20,000. The corresponding weights are 0.50, 0.30, and 0.20. These weights represent budget fractions, so they must be non-negative and sum to 1.

A numerical example: Suppose these three assets have expected returns of 2%, 6%, and 10%, standard deviations of 10%, 20%, and 30%, and zero pairwise correlations, all for the same time period. Require a portfolio return of at least 4%. Use decimal returns in the calculations (for example, 2% = 0.02).

Candidate	Weights	Expected return	Variance
A	50%, 30%, 20%	4.8%	0.0097
B	60%, 30%, 10%	4.0%	0.0081

Both allocations use the whole budget, have non-negative weights, and meet the 4% return target. B has lower variance, so (1+1)-ES would replace A with B. The population variants apply their stated survivor-selection rules and record the best allocation found. This example is explanatory only; it is not an additional experiment.

Understanding return and risk: Expected return is the estimated gain or loss based on the available data. Risk is measured here by variance, which indicates how much returns may fluctuate. Correlation describes how the returns of two assets tend to move together. Combining assets whose returns do not all move in the same way can help reduce overall portfolio risk.

What your ES program will do. Implement three ES variants that search for investment weights using the supplied asset statistics. Each allocation must use the whole budget, have no negative weights, and meet the required expected return. Each run must return the valid allocation with the lowest variance found within 30,000 objective evaluations. Compare the results of the three ES variants.

You are given asset statistics consisting of:

- Mean return vector: $r \in R^n$
- Standard deviation vector: $s \in R^n$
- Correlation matrix: $C \in R^{n \times n}$

A. Covariance Matrix Construction

Construct the symmetric $n \times n$ covariance matrix Σ using:

$$\Sigma_{ij} = s_i \cdot s_j \cdot C_{ij}$$

B. Decision Variable & Objective

A candidate solution vector $w = (w_1, w_2, ..., w_n)^T$ represents the budget fraction allocated to each asset.

- **Expected Return:** $R(w) = r^T w$
- **Portfolio Variance (Objective):** $V(w) = w^T \Sigma w$

C. Optimization Model

Minimize portfolio variance subject to long-only budget and return constraints:

$$\begin{aligned} &\text{minimize} && V(w) = w^T \Sigma w \\ &\text{subject to} && w_i \geq 0, \ i = 1, ..., n \\ & && \sum w_i = 1 \\ & && R(w) \geq R_{min} \end{aligned}$$

Note: Risk is measured here by portfolio variance. Use variance consistently as the objective in all implementations and results.

Benchmark Datasets & Required Return Settings

A. Benchmark Datasets (OR-Library)

Use all five standard OR-Library portfolio datasets. Parse the asset statistics and correlation matrices from the official text files. Official source and file format: https://people.brunel.ac.uk/~mastjjb/jeb/orlib/portinfo.html

Dataset File	Assets (n)	Correlation Records
port1.txt	31	496
port2.txt	85	3,655
port3.txt	89	4,005
port4.txt	98	4,851
port5.txt	225	25,425

Loading the data: The first line gives the number of assets, n. The next n lines give each asset's mean return and standard deviation. Each remaining record contains two asset indices, i, j, and their correlation. Store that correlation in both C_{ij} and C_{ji}. Asset indices start at 1; subtract 1 when indexing Python arrays.

B. Return Target Calculation

For each dataset, calculate the unweighted equal allocation vector $u_i = 1/n$, its return $R_{eq} = r^T u$, and the maximum individual asset mean return $r_{max} = \max_i r_i$. Define three target return levels using $\alpha \in \{0.25, 0.50, 0.75\}$:

$$R_{min}(\alpha) = (1 - \alpha) \cdot R_{eq} + \alpha \cdot r_{max}$$

Combining 5 datasets with 3 target return levels yields **15 assignment test cases** (e.g., `port1_alpha025` through `port5_alpha075`).

C. Parser Validation Checkpoint

For `port1.txt`, check your data loading against these approximate values:

- Equal allocation return: $R_{eq} \approx 0.003504064516129$
- Equal allocation variance: $V(u) \approx 0.001130937943724$
- Target return at $\alpha = 0.50$: $R_{min} \approx 0.007184532258065$

Constraint Handling & Feasibility Repair

Candidate vectors $z \in R^n$ generated during search are repaired into feasible portfolio weights w prior to objective evaluation.

Step 1: Simplex Projection

Project z onto the probability simplex so that all weights are non-negative and sum to 1:

$$q_i = \max(z_i - \theta, 0)$$

Choose θ so the weights sum to 1. A library routine for simplex projection may be used.

Step 2: Return Target Repair

Compute $R(q) = r^T q$:

- If $R(q) \geq R_{min}$, accept q directly ($w = q$).
- If $R(q) < R_{min}$, let index j attain r_{max} and form the convex combination:

$$\beta = (R_{min} - R(q)) / (r_{max} - R(q))$$

$$w = (1 - \beta) \cdot q + \beta \cdot e_j$$, Here, e_j puts all weight on asset *j*.

Feasibility Check Requirements

Every final reported solution must satisfy the following numerical tolerances:

- $w_i \geq -10^{-8}$
- $|\sum w_i - 1| \leq 10^{-8}$
- $R(w) \geq R_{min} - 10^{-8}$

Required ES Variants

Variant 1: Single-Parent (1+1)-ES

- **Population:** 1 parent and 1 mutated child. Initialize the weights with independent uniform values in [0, 1], then apply the stated repair.
- **Mutation:** Add the scalar step size times an independent standard normal value to each weight and repair the child. Retain the child only if its variance is lower than the parent's.
- **Step Size:** Start the scalar step size at $0.1 / \sqrt{n}$ and keep it in $[10^{-8}, 1.0]$. Update it per trial via the 1/5th success rule:

$$\sigma \leftarrow \sigma \cdot \exp(1/3) \quad \text{(if offspring variance is strictly lower)}$$

$$\sigma \leftarrow \sigma \cdot \exp(-1/12) \quad \text{(otherwise)}$$

Variants 2 & 3: Population ES ((μ+λ)-ES and (μ,λ)-ES)

- **Parameters:** Parent size $\mu = 10$, Offspring size $\lambda = 70$.
- **Initialization:** Initialize parent weights as in (1+1)-ES. Use the initial step size vector $0.1 / \sqrt{n}$ across all coordinates. Clamp every updated step size to $[10^{-8}, 1.0]$.
- **Recombination:** Two parents selected uniformly at random.

- Weight recombination: $w^r = (w^{(1)} + w^{(2)}) / 2$
- Step-size recombination: $\sigma_i^r = \text{sqrt}(\sigma_i^{(1)} \cdot \sigma_i^{(2)})$

- **Self-Adaptation Formula:**

$$\sigma'_i = \sigma_i^r \cdot \exp(\tau' \cdot Z_0 + \tau \cdot Z_i)$$

$$\tau' = 1 / \text{sqrt}(2n), \quad \tau = 1 / \text{sqrt}(2 \cdot \text{sqrt}(n))$$

Z_0 and Z_i are independent standard normal draws; share Z_0 across a child's coordinates. After adaptation, add the updated step size times a fresh, independent standard normal draw to each recombined weight, then repair the child. Store its updated step sizes with it.

- **Survivor Selection:**
  - **(μ+λ)-ES:** Select the μ candidates with the lowest variance from the parents and offspring combined.
  - **(μ,λ)-ES:** Select the μ candidates with the lowest variance from the offspring only.

For all three ES variants, keep a separate record (archive) of the best valid allocation found so far, including initialization. Return this allocation at the end. In (μ,λ)-ES, the archive is for reporting only and does not participate in parent selection.

Evaluation Protocol & Metrics

ES variant	Independent runs per case	Evaluations (budget) per run
(1+1)-ES	10	30,000 evals
(μ+λ)-ES	10	30,000 evals
(μ,λ)-ES	10	30,000 evals

Use the same cases, repair, 10 seeds and 30,000-evaluation budget for all three ES variants. Count initialization and every evaluated candidate, including rejected ones. Stop at the budget. For population ES, the final batch may contain fewer than 70 offspring to stay within the budget. Include these offspring when updating the best-so-far record.

The primary metric is the **final best feasible portfolio variance**. Lower variance indicates a better solution for the same dataset and return target.

For each run, record the best valid allocation, expected return, variance, and runtime. Use valid runs for variance statistics, report how many of the 10 runs were valid, and average runtime across all 10 runs.

Plots and Tables

Provide the following outputs with clear captions, axis labels, units, and consistent case identifiers. Use consistent labels for the three ES variants.

- **Summary tables:** For all 15 cases and each ES variant, report case name, number of assets, target return, best and mean variance, standard deviation, feasible-run rate, and mean runtime.
- **Convergence plot:** For `port5_alpha050` only, plot evaluations against the mean archive best variance across 10 runs. Show all three ES variants on one plot.

Analysis

Compare the three ES variants using achieved variance, variability, convergence and runtime. Discuss selection, step-size adaptation, problem size and repair.

Discuss stagnation and unsuccessful runs, including why comma selection can worsen the current population best while the archive best cannot worsen.

Limit conclusions to relative performance among the tested ES variants. These historical input statistics do not establish future investment performance.

Deliverables

Code

- The three ES implementations, common repair and validation routines, and experiment scripts.
- Scripts that reproduce every required experiment, table, and figure; the original datasets; raw per-run results; and convergence logs.
- A `README` with setup instructions, dependencies and versions, exact dataset sources, seed lists, parameter settings, and commands for one test case and the full experiment set.

Report

- Problem formulation, application context, datasets, targets, and validation.
- Briefly describe the three ES variants and the repair method. Refer to the `README` for implementation settings.
- Present the results and compare the ES variants, discussing failures and limitations.